

Dilpreet Singh Bansi

bansidilpreet@gmail.com | dilpreetbansi.github.io
linkedin.com/in/dilpreet-bansi | github.com/DilpreetBansi

EDUCATION

York University

Toronto, ON

B.Eng. in Software Engineering · GPA 3.7/4.0 · Dean's List (2021–2026), Honour Roll, Entrance Scholarship

May 2026

– **Coursework:** Machine Learning, Distributed Systems, Data Structures, Computer Networks, Software Development

EXPERIENCE

SOTI

Mississauga, ON

Software Developer, AI/ML

May 2026 – Present

- Lead natural-language app generation in **SOTI Snap**, the company's enterprise low-code platform, turning plain-English descriptions into production-ready iOS & Android apps and cutting app build time **80%**
- Architect **LLM** pipelines combining **RAG**, prompt engineering, and **agentic workflows** that turn requirements into working UI, data models, and backend logic across **100+ languages**
- Work 1:1 with the **CEO** and executives on the AI product roadmap; shipped a working prototype in the first month
- Ship AI features in **C#.NET** and **Angular** under Agile/Scrum, backed by automated tests, performance benchmarks, and LLM output evaluation

Tegra Lumber

Cambridge, ON

Software Engineer, Full-Stack

Jan 2024 – May 2026

- As **sole engineer**, built the company's operations platform end to end, starting with inventory and yard management (**React/TypeScript, Node/Express, PostgreSQL**); cut cycle-count time **45%** and stockouts **28%**
- Built a pricing and quoting service (board-foot conversion, tiered/contract pricing, fuel surcharges) that cut quote turnaround from **6 hours to 5 minutes**
- Launched a customer order portal with real-time availability, pricing, and pickup scheduling; moved **14%** of counter orders online within six weeks and cut peak-hour phone traffic **35%**
- Set up CI/CD (**GitHub Actions, Docker**, automated migrations) with blue-green deploys; cut deploy time 15 → 2 min
- Raised test coverage to **82%** (Jest, Playwright), catching 14 regressions before release across 14 releases; reached **93% adoption** among staff new to software

PROJECTS

llama-wgsl | JavaScript, WebGPU, WGSL

- Runs SmolLM2 on the user's GPU inside the browser with hand-written WGSL compute kernels (fused QKV and SwiGLU matvecs, grouped-query attention, RoPE, KV cache); no server, no dependencies
- Streams safetensors weights straight to GPU memory as packed bf16 and reuses cached prefixes across chat turns; logits match Hugging Face **transformers** within **5e-5** and greedy output is identical token for token

FlashServe | Python, PyTorch, FastAPI

- Llama inference engine for Hugging Face checkpoints: FlashAttention-style tiled attention, KV-cache decoding (**6.1×** faster than recompute), speculative decoding, a paged KV cache, and an OpenAI-compatible streaming API

TinyTrain | Python, PyTorch

- Data, tensor (Megatron-style), and pipeline (GPipe) parallelism from scratch on torch.distributed, with a hand-written ring all-reduce; tests prove each strategy matches single-process training

NeuronLens | Python, PyTorch

- Interpretability toolkit that reproduces published GPT-2 results: induction heads, IOI name movers, causal tracing

AgentForge | Python, asyncio

- Planner-executor-verifier agent framework with OpenAI-compatible tool calling and verify-and-retry loops

LatencyArb | C++17, Python

- C++17 execution core (lock-free SPSC queue at **227M** msgs/s, clean under ThreadSanitizer) plus a Python pairs-trading backtester with cointegration tests and Kalman-filter hedge ratios

TECHNICAL SKILLS

Languages: Python, TypeScript/JavaScript, C#, C++, SQL, Java, WGSL

AI/ML: PyTorch, Hugging Face, LLM inference, RAG, agents, evals, LangChain/LangGraph, OpenAI API, WebGPU

Backend & Cloud: FastAPI, Node.js, .NET, PostgreSQL, Redis, MongoDB, Docker, Kubernetes, AWS, GCP, CI/CD

Frontend & Testing: React, Angular, HTML/CSS, Jest, Playwright, Pytest